Operating Systems — Assignment 10

Memory Management, Virtual Memory, Paging and Page Replacement in xv6

Introduction

This assignment extends the xv6 kernel (x86, two-level paging) with three new memory-related system calls and implements the classic page replacement algorithms as a user program. Four new system calls were added following the standard xv6 plumbing: numbers in `syscall.h` (22–25), dispatch entries in `syscall.c`, handlers in `sysproc.c`, prototypes in `user.h`, and stubs in `usys.S` (`SYSCALL(memstats)`, `SYSCALL(vtop)`, `SYSCALL(pageinfo)`, `SYSCALL(setreadonly)`). Every modification is commented in the source.

Q1 — Tracking physical memory allocation (`memstats`)

`kalloc.c` now maintains three counters, updated inside the allocator lock: `total_allocated_pages` and `total_freed_pages` count `kalloc()`/`kfree()` events, and `free_pages` tracks the exact length of the free list. Counting of events starts at the end of `kinit2()` so that the boot-time `freerange()` calls (which push every physical page onto the free list) do not distort the statistics. The new system call `int memstats(void)` prints total allocated, total freed, currently allocated (= allocated − freed) and available pages (and available memory in KB). The user program `memtest.c` displays the statistics, allocates 4 pages (16 KB) with `sbrk()`, displays them again, releases the memory with a negative `sbrk()` and displays the final state. **Observation:** after allocation, currently allocated pages rise by the pages used for the process growth; after release they return to (approximately) the earlier value, confirming that `deallocuvm()` returns the frames to `kfree()`.

Q2 — Virtual-to-physical translation (`vtop`)

A helper `getpte()` in `vm.c` wraps `walkpgdir(pgdir, va, 0)` so no page table is created for unmapped addresses. `int vtop(int va)` fetches the PTE of the calling process, checks the Present bit (`PTE_P`), and returns `PTE_ADDR(pte) + (va & 0xFFF)` — frame base plus page offset — or −1 for an unmapped/not-present page. `vttest.c` translates heap, data, code and stack addresses and two invalid addresses. **Observation:** addresses inside the same page differ only in the offset bits, consecutive heap pages map to different frames, and invalid addresses (e.g. 0x10000000) correctly return −1.

Q3 — Page information and protection (`pageinfo`, `setreadonly`)

`int pageinfo(int va)` reports whether a page is mapped, its physical frame number and the P, W and U flag bits of its PTE. `int setreadonly(int va)` (clearing logic in `setreadonly_vm()` in `vm.c`) clears `PTE_W` and reloads `%cr3` to flush the TLB so the change takes effect immediately. `pagetest.c` allocates a page, shows P=1, W=1, U=1, writes to it successfully, makes it read-only (W becomes 0), and writes again. **Observation:** the second write raises a page fault (trap 14) and xv6 kills the process, demonstrating that page-level protection is enforced by hardware once the PTE bit is cleared.

Q4 — Page replacement: FIFO, LRU, Optimal

`pagereplace.c` simulates all three algorithms on the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 (20 references), printing the frame state and HIT/FAULT for every reference with 3 frames, and a comparison for 3, 4 and 5 frames:

Frames	FIFO faults	LRU faults	Optimal faults
3	15	12	9
4	10	8	8
5	9	7	7

Observation: Optimal gives the fewest faults with 3 frames and is never worse than the others, but it needs future knowledge, so it is only a benchmark. LRU is the best practically implementable algorithm here (it even matches Optimal at 4–5 frames); FIFO is the worst for this string because it ignores how recently or frequently a page is used. More frames reduce faults for all three algorithms, as expected.

Conclusion

The assignment connected the three layers of xv6 memory management — the physical page allocator, the per-process page tables, and replacement policy theory — with working, tested code: allocator statistics are observable from user space, address translation and protection bits can be inspected and modified through

system calls, and the replacement simulation reproduces the textbook results for the given reference string.